MANUAL TÉCNICO

Índice

Introducción 1
Arquitectura 2
Modelo de Datos 3
Servicios REST del Backend 4
Seguridad 6
Instalación y Despliegue 6
Mantenimiento 10

Introducción

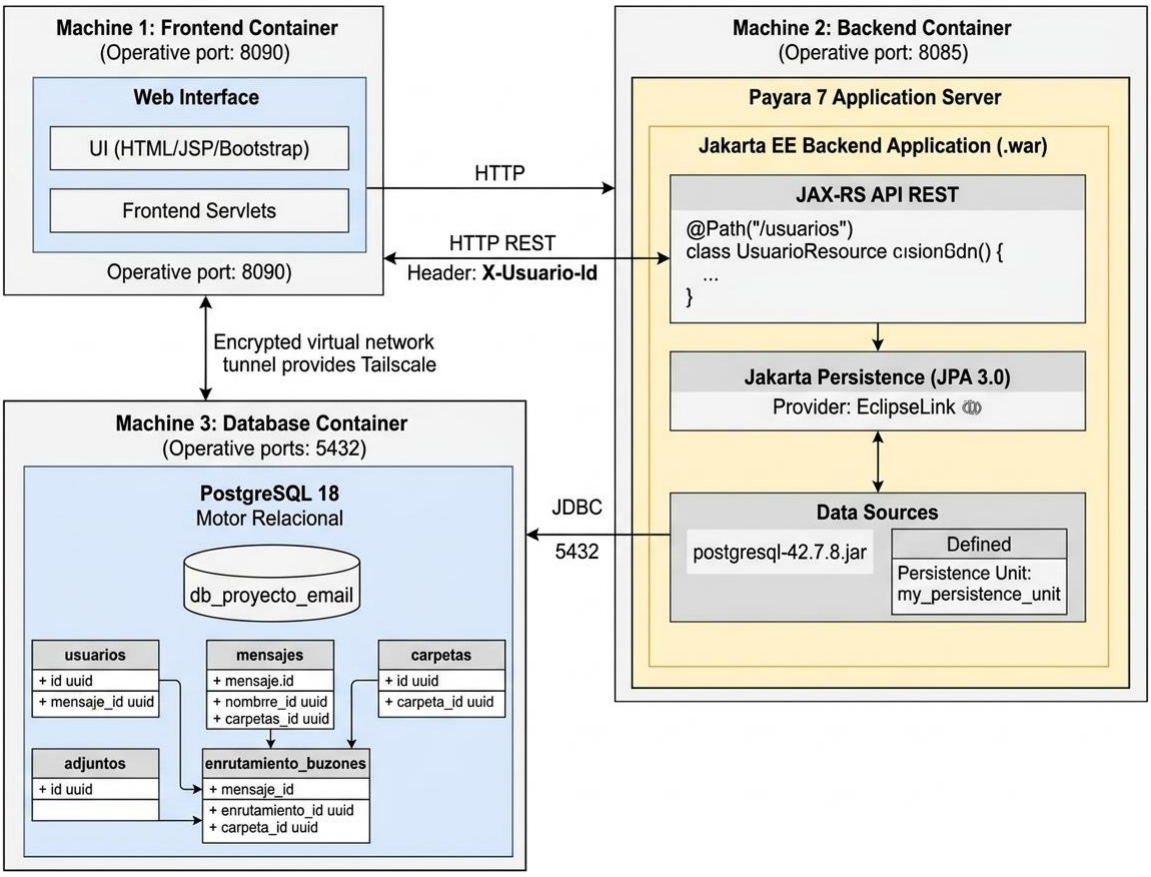
El propósito de este sistema es proveer una plataforma web de correo electrónico corporativo que permita la gestión, envío y recepción de mensajería interna. En esta versión entregada, el alcance abarca la autenticación segura, la gestión de carpetas personalizadas, el flujo de mensajería asíncrona y la administración centralizada de registros.

Las tecnologías y versiones exactas compiladas en el proyecto son:

- **Java SE 17 / Jakarta EE 11 (API 11.0.0-M1):** Plataforma base de desarrollo y ejecución de servicios empresariales.
- **EclipseLink (JPA 3.0):** Proveedor ORM configurado como motor de persistencia de datos.
- **Payara 7:** Servidor de aplicaciones empresariales donde se despliega el backend.
- **PostgreSQL 18:** Motor relacional de base de datos conectado mediante el controlador oficial **postgresql-42.7.8.jar**.
- **Apache NetBeans:** Entorno de desarrollo integrado (IDE) manejado mediante el modelo de objetos de Maven.

- **Formato de Empaquetado (war):** Archivo web comprimido (backend-correo-1.0-SNAPSHOT.war) generado para la distribución y despliegue.
- **Bootstrap / HTML:** Framework de diseño responsivo implementado para la maquetación de la interfaz del frontend

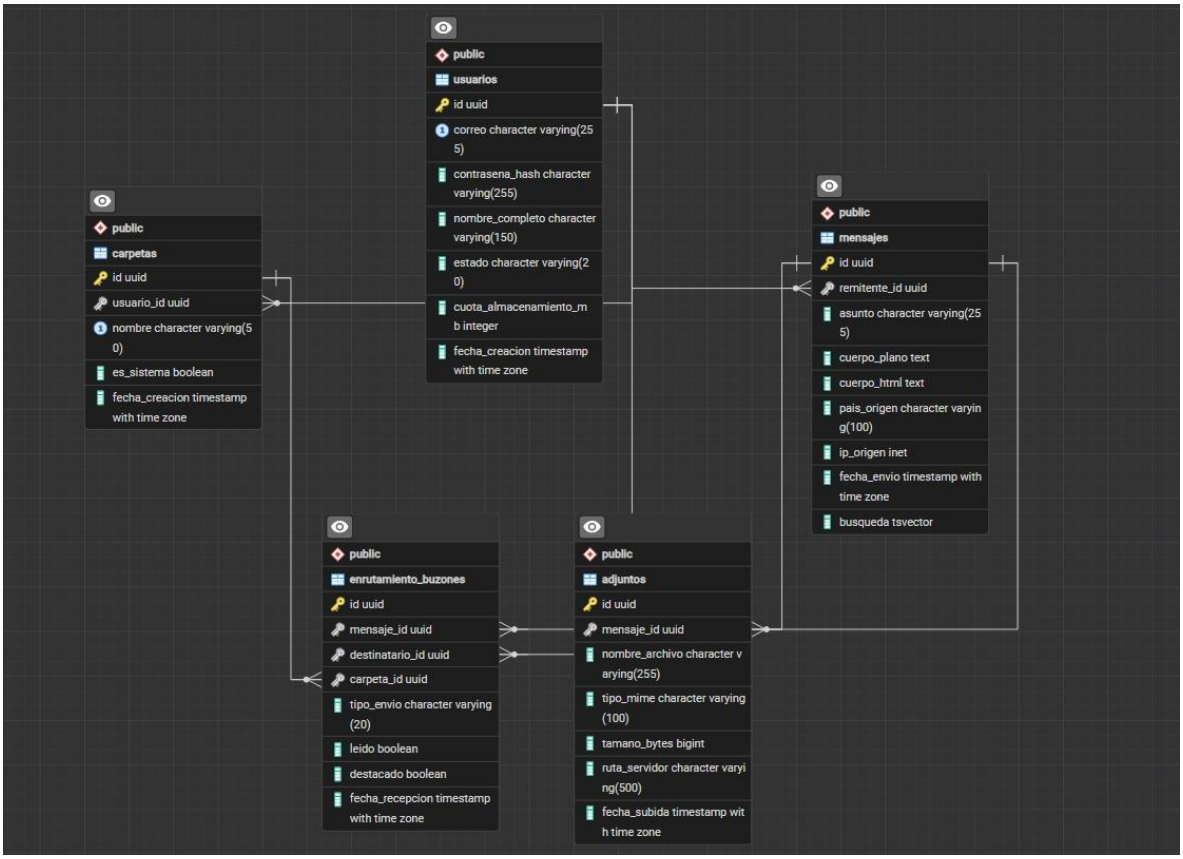
Arquitectura



1. **Capa de Presentación (Frontend):** Vistas web diseñadas con Bootstrap, controladas mediante Servlets y JSP en el puerto **8090**.
2. **Capa de Negocio (Backend):** API REST impulsada por la especificación JAX-RS sobre Payara 7 en el puerto **8085**.
3. **Capa de Datos:** Persistencia relacional en PostgreSQL (Puerto 5432) interactuando dinámicamente con EclipseLink.

- **Comunicación Frontend-Backend:** El frontend intercepta las acciones en la interfaz y genera peticiones HTTP orientadas a servicios REST. Para identificar de manera unívoca al usuario autenticado, el frontend inyecta obligatoriamente el identificador único en el encabezado de cada petición HTTP bajo la clave **X-Usuario-Id**.
- **Capa de Datos:** El almacenamiento se gestiona en la base de datos db_proyecto_email estructurada en 5 tablas principales ligadas de manera lógica (usuarios, carpetas, mensajes, adjuntos y enrutamiento_buzones).

Modelo de Datos



1. **usuarios** (Cuentas activas en la plataforma):
    - *Llave Primaria:* id (uuid)
    - *Atributos:* correo (varchar 255), contrasena_hash (varchar 255), nombre_completo (varchar 150), estado (varchar 20), cuota_almacenamiento_mb (integer), fecha_creacion (timestamp with time zone).
2. **carpetas** (Secciones de organización de buzones):

- *Llave Primaria:* id (uuid)
- *Llave Foránea:* usuario_id (uuid) -> Referencia a usuarios(id).
- *Atributos:* nombre (varchar 50), es_sistema (boolean), fecha_creacion (timestamp with time zone).

3. **mensajes** (Estructura física del correo electrónico):
   - *Llave Primaria:* id (uuid)
   - *Llave Foránea:* remitente_id (uuid) -> Referencia a usuarios(id).
   - *Atributos:* asunto (varchar 255), cuerpo_plano (text), cuerpo_html (text), pais_origen (varchar 100), ip_origen (inet), fecha_envio (timestamp with time zone), busqueda (tsvector para búsquedas de texto completo).
4. **adjuntos** (Archivos binarios enlazados a los mensajes):
   - *Llave Primaria:* id (uuid)
   - *Llave Foránea:* mensaje_id (uuid) -> Referencia a mensajes(id).
   - *Atributos:* nombre_archivo (varchar 255), tipo_mime (varchar 100), tamano_bytes (bigint), ruta_servidor (varchar 500), fecha_subida (timestamp with time zone).
5. **enrutamiento_buzones** (Gestión e indexación de destinos de entrega):
   - *Llave Primaria:* id (uuid)
   - *Llaves Foráneas:* mensaje_id (uuid) -> mensajes(id); destinatario_id (uuid) -> usuarios(id); carpeta_id (uuid) -> carpetas(id).
   - *Atributos:* tipo_envio (varchar 20), leido (boolean), destacado (boolean), fecha_recepcion (timestamp with time zone).

Servicios REST del Backend

El componente UsuarioResource.java publica los servicios de la API bajo el recurso raíz @Path("/usuarios") interactuando bajo el formato application/json.

Catálogo de Endpoints de Usuarios [1]

Método HTTP	Ruta del Endpoint	Parámetro Requerido	Cuerpo Esperado (JSON Body)	Respuesta de Éxito

		(Header / Path)		
POST	/usuarios	Ninguno *(Público)*	Propiedades del objeto Usuario	201 Created + JSON Usuario
GET	/usuarios	Header: X-Usuario-Id *(Admin)*	Ninguno	200 OK + Lista Usuarios []
GET	/usuarios/{id}	Path: id (UUID) Header: X-Usuario-Id *(Admin)*	Ninguno	200 OK + JSON de la entidad
PUT	/usuarios/{id}	Path: id (UUID) Header: X-Usuario-Id *(Admin)*	Campos a actualizar (nombreCompleto, estado, cuotaAlmacenamientoMb)	200 OK + JSON actualizado
DELETE	/usuarios/{id}	Path: id (UUID) Header: X-Usuario-Id *(Admin)*	Ninguno	204 No Content *(Borrado lógico)*
POST	/usuarios/login	Ninguno *(Público)*	{ "correo": "...", "contrasena": "..." }	200 OK + Sesión (id, correo, esAdministrador)
POST	/usuarios/recuperar	Ninguno *(Público)*	{ "correo": "..." }	200 OK + Mensaje de envío

Códigos de Error Controlados [1]

- **400 Bad Request:** Datos de actualización inválidos (cuotas <= 0, estados incorrectos o intento de autoborrado).

- **41 Unauthorized:** Credenciales de login erróneas o token de sesión inválido.
- **403 Forbidden:** Acceso denegado. Ocurre si esAdministrador(solicitanteId) evalúa como falso.
- **404 Not Found:** El UUID proporcionado no coincide con ningún registro en PostgreSQL.
- **500 Internal Server Error:** Excepción capturada (SQLException), reflejando fallas físicas en la base de datos o en Payara.

Seguridad

- **Protección de Registro Abierto:** El método de alta ignora de forma explícita el parámetro es_administrador enviado por clientes externos, evitando la creación ilícita de administradores autónomos.
- **Validación de Roles:** Las operaciones críticas exigen la cabecera X-Usuario-Id. El sistema evalúa el ID y restringe accesos no autorizados mediante código nativo de control de excepciones.
- **Restricción de Autoborrado Logical:** El método DELETE compara el parámetro id contra el header X-Usuario-Id. Si coinciden, interrumpe el flujo con un error 400 para proteger la sesión del administrador activo.
- **Resguardo Criptográfico:** La clase utilitaria PasswordUtil.java procesa la clave recibida mediante algoritmos de hashing seguro y salado (*salting*) antes de impactar el campo contrasena_hash.
- **Ofuscación ante Intrusión:** El endpoint /usuarios/recuperar emite la misma respuesta de éxito existala cuenta o no en el sistema, mitigando técnicas de enumeración de usuarios.

Instalación y Despliegue

Requisitos del entorno

Para la correcta ejecución y despliegue del sistema de correo distribuido, los entornos deben cumplir con los siguientes componentes de software y hardware mínimos:

- **Sistema Operativo:** Windows 10/11 con soporte para **WSL2** (Windows Subsystem for Linux) o distribuciones Linux basadas en Debian/RHEL.

- **Entorno de Virtualización: Docker Desktop** (versión v24.0 o superior) habilitado en el equipo anfitrión.
- **Conectividad Virtual VPN:** Cliente **Tailscale** instalado en cada una de las 3 máquinas del entorno distribuido.
- **Servidor de Aplicaciones (Desarrollo Local): Payara Server 7** configurado bajo el entorno de ejecución **Java SE 17**.
- **Controlador de Base de Datos:** Archivo binario **postgresql-42.7.8.jar** para la conexión por puente JDBC.
- **Base de Datos Nativa:** Instancia de **PostgreSQL 18** configurada para escuchar peticiones en el puerto estándar **5432**.

Pasos para instalación con docker

El despliegue de producción segmenta el sistema en tres contenedores aislados de Docker correspondientes a las capas del proyecto (Frontend, Backend, Base de Datos):

1. **Configurar Variables de Entorno:** Crear un archivo .env en la raíz del proyecto para externalizar credenciales físicas.
2. **Unirse al Entorno de Red (Tailscale):**
    - Cierre cualquier sesión personal en la aplicación de Tailscale.
    - Abra el enlace web de invitación del proyecto y presione **"Connect"** en la pantalla *Connect device* tras elegir el correo asignado a la red del anfitrión.
3. **Compilar el Artefacto:** Generar el archivo web de distribución (backend-correo-1.0-SNAPSHOT.war) a través de Maven mediante el IDE Apache NetBeans.
4. **Ejecutar el Archivo por Lotes:** En la consola del sistema, situarse en el directorio raíz y ejecutar el script automatizado:

bash

iniciar.bat

Usa el código con precaución.

(Alternativamente, levantar la infraestructura con el comando nativo de Docker: docker-compose up -d --build).

5. **Verificación de Puertos Operativos:** Validar en la interfaz de Docker Desktop que el contenedor de Frontend se exponga en el puerto **8090** y el contenedor de Backend (Payara 7) responda en el puerto **8085**.
6. **Cierre Seguro del Sistema:** Para dar de baja los contenedores de forma limpia, ejecutar el script detener.bat o el comando docker-compose down.

Pasos para instalación tradicional (sin docker)

Esta configuración se emplea exclusivamente para depuración monolítica en un único equipo de desarrollo local:

1. **Carga del Driver en GlassFish:** Mover físicamente el archivo **postgresql-42.7.8.jar** a la ruta de extensiones del dominio activo de Payara: C:\payara7\glassfish\domains\domain1\lib\ext
2. **Inicialización de la Base de Datos:** Levantar el servicio local de PostgreSQL, acceder mediante un cliente (como pgAdmin) y crear la base de datos bajo el nombre exacto de **db_proyecto_email**.
3. **Configuración del Descriptor JPA (persistence.xml):** Validar que el archivo descriptor interno bajo la ruta META-INF/persistence.xml mantenga declarada la unidad de persistencia **my_persistence_unit** apuntando al proveedor **EclipseLink** y los accesos locales:
    - *URL:* jdbc:postgresql://localhost:5432/db_proyecto_email
    - *Usuario:* postgres | *Contraseña:* 1234
4. **Despliegue del WAR:** Iniciar el servidor local Payara 7. Ingresar a la consola web de administración (http://localhost:4848), dirigirse al apartado **Applications**, presionar *Deploy* y cargar el archivo empaquetado backend-correo-1.0-SNAPSHOT.war.
5. **Prueba de Disponibilidad:** Validar el correcto mapeo abriendo el navegador web en la ruta: http://localhost:8080/backend-correo-1.0-SNAPSHOT/resources/usuarios.

Pruebas

Plan de pruebas

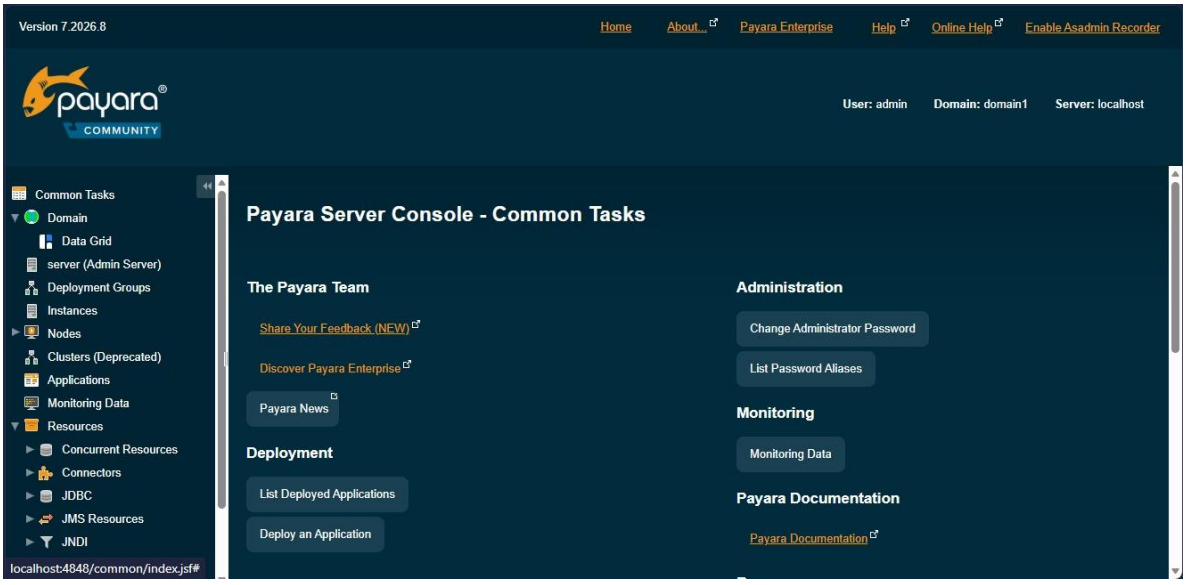
El plan de pruebas tiene como objetivo verificar la correcta integración de los componentes del sistema y garantizar la comunicación en el entorno distribuido. Los componentes específicos que se someten a evaluación son:

- **Componente de Persistencia (my_persistence_unit):** Validación del canal de datos gestionado por EclipseLink hacia PostgreSQL.
- **Controlador de Servicios REST (UsuarioResource.java):** Validación de los métodos de negocio expuestos bajo la ruta raíz /usuarios.
- **Filtro de Seguridad (Autorizacion.java):** Validación de la recepción e interceptación del encabezado HTTP X-Usuario-Id.
- **Servicio de Cifrado (PasswordUtil.java):** Verificación del correcto procesamiento de hashing de credenciales durante el alta.

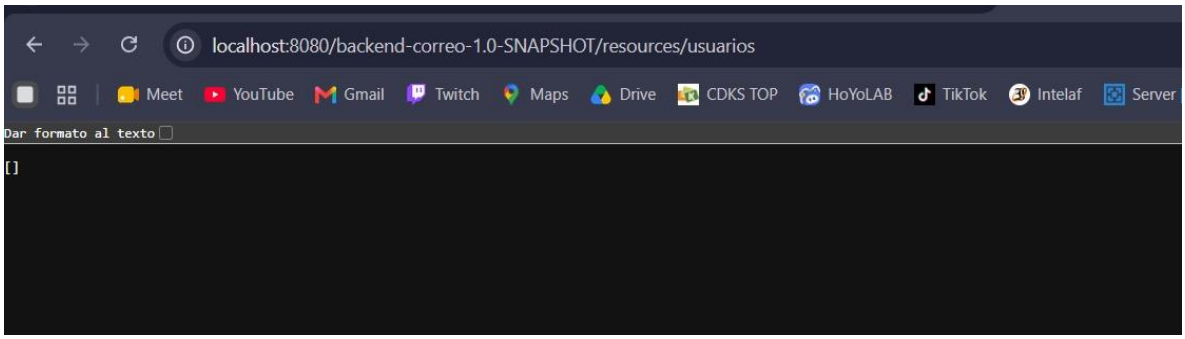
Ejecución de pruebas

La ejecución de las pruebas se realiza utilizando herramientas de simulación y entornos de gestión integrados en el ecosistema de desarrollo:

- **Consola de Administración de Payara (Puerto 4848):** Herramienta web utilizada para lanzar peticiones de diagnóstico directo al pool de recursos de datos.



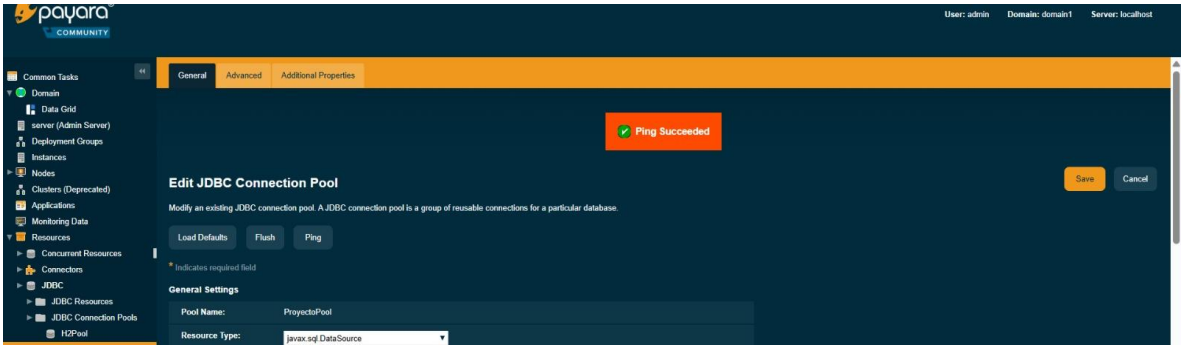
- **Navegador Web (Chrome/Edge):** Cliente utilizado para verificar de manera directa la accesibilidad de los recursos públicos de la API mediante peticiones HTTP GET.



Evidencia de pruebas

El éxito en el aseguramiento de la calidad del software se valida mediante los siguientes resultados y respuestas del sistema:

- **Mensaje "Ping Succeeded":** Respuesta gráfica emitida de manera exitosa por el servidor Payara 7 que confirma que la librería postgresql-42.7.8.jar logró abrir el canal hacia la base de datos db_proyecto_email.



Mantenimiento

- **Rotación de Credenciales:** Toda modificación a las credenciales nativas de la base de datos exige la edición sincronizada del archivo de persistencia XML, las variables del archivo .env del contenedor y el script de arranque post-boot.asadmin de Payara.
- **Respaldos de Datos Relacionales:** Ejecución rutinaria en el servidor de base de datos del utilitario nativo:

bash

```
pg_dump -U postgres db_proyecto_email > respaldo_correo.sql
```

- **Solución a Bloqueo en Red (Tailscale):** Si aparece el error *"Se requiere aprobación del usuario"*, se debe a un inicio de sesión prematuro. La solución requiere cerrar sesión en la aplicación, abrir el enlace del anfitrión y conectarse desde la ventana web oficial.